

Data Structures:

- Line struct:
 - size_t length,
 - Int representing the length of the corrupted line
 - char *orig_cont,
 - Holds corrupted line contents
 - char *clean_cont,
 - Holds digits from lines
 - Char *raw
 - Holds final, restored P5 format text
 - Hanson Atom nondigits
 - Atomized version of non-digit text in the line
- Hanson List called **lines**, of pointers to Line structs
 - List_new (int hint)
 - List_push(list, line struct *) ---- pushes to back of list
 - List_free (*list) ----remember, we need to empty all the structs first
 - List_map(list, void apply(void **x, void *cl), void *cl)
- Hanson Table called **atom_to_struct** where key = atoms, value = pointers to Line structs
 - Table_new(int hint, cmp, hash) *where cmp and hash are functions used to map keys to values. We will just call Table_new(0, NULL, NULL) so that it just uses the default functions, since our keys are atoms, which is what the default supports.*
 - Table_put(*table, *atom, *line struct)
 - Table_map(table, apply, cl) *will be used to deallocate keys and values*
 - Where apply is a function to do on all the values and cl is an optional extra pointer that can be provided and used by apply. Otherwise pass NULL
 - Table_free(*table) *will be used to deallocate *table and set it to null pointer*
- Hanson Atoms
 - Atom_new(char *nondigit_str, int length_of_nondigit_str)
- Arrays
 - Sizeof(array)

Implementation Plan

Creating Project Files

1. Create the .c file for your restoration program. Write a main function that spits out the ubiquitous "Hello World!" greeting. Compile and run. **Time: 10 minutes**
2. Create the .c file that will hold your readline implementation. Move your "Hello World!" greeting from the main function in restoration to your readline function and call readline from main. Compile and run this code. **Time: 5 minutes**

Checking that Restoration can open files & behave accordingly

3. In restoration, Check if argc > 2, if so then return a runtime error. If argc = 2 open (and add a matching close for) the given file. If argc = 1, set file pointer to stdin. If nothing is provided in stdin, Checked Runtime error. See idioms.
If argc = 2, check if the file opened correctly. If argc = 1, expect it to read the standard output or send runtime error if standard input is empty. Time: 10 min

Reading Files with readline

4. Inside readline, check if either arg (file or datapp) are equal to NULL and if so terminate the program with a runtime error. **1. Give readline a file = NULL, expect back a runtime error. 2. Give readline with a **datapp set to NULL, expect a runtime error. Time: 10 minutes**
5. Check if we are at the end of the file, if we are then set datapp to NULL and return 0. **1. Give readline an empty file. Expect back the number 0 and datapp to be set to NULL. 2. Give readline a file where there are valid characters in the line. Expect back any int other than 0. Time: 5 minutes**
6. Allocate an array (char *line_arr) of size 1000 then check whether the memory allocation worked, if it did not then terminate with a Checked Runtime Error
Give readline a file where neither argument is = to NULL, attempt to read the first character and send it to **datapp, then print the value of **datapp, expect back a printf of the first character of the line. Time: 10 minutes
7. Make a while loop to read in characters from a line until a \n is hit, or we've read in 999 characters (make an integer outside of loop to keep track of num chars read, and a char next_char to hold 1 char at a time). **Inside loop, print each character that is read and increment num_chars. Make a file Threelines.txt with lines of length 1, 100, 1000, (including \n). Call readline three times. Expect all three lines characters (note that the first line is just a \n) to be printed. Time: 15 minutes**
8. Make an if statement to check if num_chars == 999. If so, put a \n in last spot of array, and make a while loop to continue reading from file into next_char, until next_char == \n. If not, add a \0 char to line_arr[num_chars]. **Make a file called Over1000.txt with 3 lines that are all over 1000 characters (and all ending with a \n). Call readline 3 times. Expect the first 999 characters of each line to be printed. Time: 15 minutes**

9. Return to the while loop from step 7. Remove print statement inside. Then, add next_char to line_arr[num_chars], increment num_chars, and then read in next character from file into next_char. Outside the while loop: for i in num_chars, print out array[i]. Make a file Fourlines.txt, copy of Threelines.txt but with one of the oversize lines from step 8. Expect to see whole file printed, except excess characters in long line. Time: 10 minutes
10. Dereference datapp once, to get *datapp (aka datap), and set that to point to the first elem of lines_arr. (*datapp = lines_arr;) In restoration, declare char *datap; Call readaline(fourlines_fp, &datap). Then, in restoration, printf datap. Expect the first line of the file to be printed. Time: 10 min
11. Make a while loop in restoration that runs until EOF (When readaline returns 0). Call readaline like above test, then print out what has been stored in datap after readaline was called, followed by a newline. Run program on files provided by course staff, and all our other test files. All characters from the file should be printed, except for those on lines > 1000 characters (including \n character) Time: 20 minutes

Restoring corrupted img

12. In restoration, define the line struct (see data structures) called line that holds an int of size_t called length, three pointers to char arrays called org_cont, clean_cont, and raw, and an atom called non_digit. Initialize clean_cont and raw to empty arrays of length 1000. Define an instance of the line struct, print out its data. Time: 15 minutes
 13. Create new Hanson List called lines of type line struct *
Try using List_push to push the line struct we made as a test above onto lines. To test deleting the memory we've just allocated, free all the memory within the struct, then call List_free on lines. Run with valgrind to see if memory is getting cleaned up. Time: 20 minutes
 14. Add to while loop from step 11: Count how many lines have been read with an integer num_lines. Also, after each readaline call, create new line struct *new_line. Initialize its length (what readaline returns) and orig_contents (what datap points to). Then push the struct onto the back of lines: lines.addhi(new_line). In restoration, after this while loop runs, all lines from the file should be represented as line structs in our Hanson List. To confirm that all data was read in correctly, map a print function onto the list that prints each struct's length and then its orig_contents. Again, run program on input files, checking that all data from the file is represented in a struct. Time: 30 minutes
 15. After lines is all set up, we are going to map a function as defined below called cont_deconstructor onto each Line struct in lines.
- void cont_deconstructor (Line struct *line, void *cl)**
(Where we pass a pointer to struct cl{Table *table, Atom *infusion_seq};)

16. Loop through each character in line's orig_cont. **Print each character. Run program on a small test file. Expect that every character in the file is printed.**
Time: 5 mins
17. create a local variable char *junk. In the loop, check whether each character is a digit. If a char is a digit then add it to the line's clean_cont. If it is not a digit then add a whitespace to clean_cont and add the current char to junk. **Provide a file with a line 10a29b73cd93e and then check clean_cont and junk (print junk in loop). Expected output for clean_cont should be 10 29 73 93, and junk string should be abcde.** Time: 15 min
18. After each run of the loop, initialize current struct's nondigit to Atom_new(junk_str, junk_str_length) **Create test file with two lines, that each have the same junk. Run program on this file. Include if (lines[0].nondigit == lines[1].nondigit) printf("atoms behaving correctly"). Create another test file with two lines that do not have the same junk. Test that their atoms are not equal.**
Time: 15 minutes
19. **Outside of cont_destructor:** Add before function call: Create Hanson table called atom_to_struct. Time: 5 mins
20. **Back inside cont_destructor:** Outside the loop, directly after nondigit is initialized, add a key-value pair to the table where the key is nondigit and its value is a pointer to the current line struct. **Create a test file with 4 unique infusion sequences. Run program on this file and confirm that outside of for loops that Table_length is 4.** Time: 15 mins
21. If Table_put does not return NULL, then it returns a pointer to a line struct. In this case, we save a pointer to that struct's atom to infusion_seq. **Create a test file where two lines have the same infusion seq and the rest are unique. Run program. Infusion_seq should contain the same atom as those of the matching lines. Make a loop after call to map, to print out the orig_cont of all lines with nondigit == infusion_seq. Expected output: all lines with that matching seq.** Time: 15 minutes.

End of Cont_destructor

Beginning of: void convert_to_raw (Line struct *, NULL)

22. Map convert_to_raw onto Hanson list lines. **Have convert_to_raw just print out what's in clean_cont. Run on small test files.** Time: 10 minutes
23. convert_to_raw checks if line.non_digit == *infusion_seq, then if so loops through clean_cont and converts the integers to ascii and adds those ascii chars to raw. **Create a test line in a file where the clean_cont is 45 55 62. After map is called, print the line's raw. Expected output for raw is "-7>"** Time: 20 minutes

End of convert_to_raw

In restoration main:

24. Create two integer variables width and height. Assign sizeof(lines[0].clean_cont) to width. Assign num_lines to height. **Print width and height. Expected value is that of sizeof(clean_cont) and num_lines. Time: 5 mins**

25. Print header- P5, width, height, and 255 (with appropriate spacing). Then map helper function print_real onto lines. **First just have print_real print each line's orig_cont. Time: 10 mins**

void print_real(Line struct *line, Atom *infusion_seq)

26. Check if this struct's nondigit == infusion_seq. If so, printf their char *raw. **Try on different files. One without a matching infusion_seq. One where every line has the correct infusion sequence. Finally, a file where only one struct matches. Time: 10 minutes**

27. Use free to delete all the allocated memory on the heap. This includes:

- For each struct: length, clean_cont, orig_cont, raw.

map a function: void free_line_data (Line struct *, NULL)

- Hanson data structures: lines, atom_to_struct, struct cl

Run with valgrind on provided input files from course staff, and the ones we've referenced before. Time: 30 minutes

Note: for ease, we've written lines[i] as if we can access things from the Hanson list this way. We understand that in code, this will instead use List_get(lines, i).